Gitの仕組みと用語

慶應義塾大学理工学部物理情報工学科
渡辺

はじめに

Gitは簡単ではない

とりあえず覚えたいコマンド

ローカルリポジトリの操作

```
git init
git add
git commit
```

状態や歴史の確認

```
git status
git diff
git log
```

ブランチの操作

```
git switch
git merge
git rebase
```

リモートとのやりとり

```
git clone
git remote
git fetch
```

「とりあえず」でも
こんなにある

git log のオプション(抜粋)

--abbrev-commit
--abbrev[=<n>]
--after=<date>
--all
--all-match
--alternate-refs
--ancestry-path
--anchored=<text>
--author-date-order
--author=<pattern>
--basic-regexp
--before=<date>
--binary
--bisect
--boundary
--branches[=<pattern>]
--break-rewrites[=[<n>][/<m>]]
--cc
--check
--cherry
--cherry-mark
--cherry-pick
--children
--color-moved-ws=<modes>
--color-moved[=<mode>]
--color-words[=<regex>]
--color[=<when>]
--combined-all-paths
--committer=<pattern>
--compact-summary
--cumulative
--date-order
--date=<format>
--decorate-refs-exclude=<pattern>
--decorate-refs=<pattern>
--decorate[=short|full|auto|no]
--dense
--dense
--diff-merges=[options]
--dirstat-by-file[=<param1,param2>…]
--dirstat[=<param1,param2,…>]
--do-walk
--dst-prefix=<prefix>
--encoding=<encoding>
--exclude=<glob-pattern>
--expand-tabs=<n>
--ext-diff
--extended-regexp
--find-copies-harder
--find-copies[=<n>]
--find-object=<object-id>
--find-renames[=<n>]
--first-parent
--fixed-strings
--follow
--format=<format>
--full-diff
--full-history
--full-index
--function-context
--glob=<glob-pattern>
--graph
--grep-reflog=<pattern>
--grep=<pattern>
--histogram
--ignore-all-space
--ignore-blank-lines
--ignore-cr-at-eol
--ignore-matching-lines=<regex>
--ignore-missing
--ignore-space-at-eol
--ignore-space-change
--ignore-submodules[=<when>]
--indent-heuristic
--inter-hunk-context=<lines>
--invert-grep
--irreversible-delete
--ita-invisible-in-index
--left-only
--left-right
--line-prefix=<prefix>
--log-size
--max-count=<number>
--max-parents=<number>
--merge
--merges
--min-parents=<number>
--minimal
--name-only
--name-status
--not
--notes[=<ref>]
--numstat
--oneline
--output-indicator-context=<char>
--output-indicator-new=<char>
--output-indicator-old=<char>
--output=<file>
--parents
--patch
--patch-with-raw
--patch-with-stat
--patience
--perl-regexp
--pickaxe-all
--pickaxe-regex
--pretty[=<format>]
--raw
--reflog
--regexp-ignore-case
--relative-date
--relative[=<path>]
--remotes[=<pattern>]
--remove-empty
--reverse
--right-only
--rotate-to=<file>
--shortstat
--show-linear-break[=<barrier>]
--show-notes[=<ref>]
--show-pulls
--show-pulls
--show-signature
--simplify-by-decoration
--simplify-merges
--since=<date>
--single-worktree
--skip-to=<file>
--skip=<number>
--source
--sparse
--src-prefix=<prefix>
--stat[=<width>[,<name-width>[,<count>]]]
--stdin
--submodule[=<format>]
--summary
--tags[=<pattern>]
--text
--textconv
--topo-order
--unified=<n>
--until=<date>
--walk-reflogs
--word-diff-regex=<regex>
--word-diff[=<mode>]
--ws-error-highlight=<kind>

はじめに

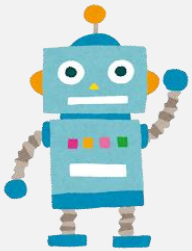
Gitは簡単ではない

Gitにはコマンドが多い
Gitはユニークな設計思想を持つ
Gitは使い方の自由度が高い(人によって流儀が異なる)
Gitに慣れていないとトラブルへの対処が難しい

コマンドを覚える必要あるの？

AIを使えば、Gitの細かいコマンドとか覚えなくていいんじゃない？

おそらく今後、GitはAIエージェントを通じて操作することになる。しかし、トラブル防止や、トラブルからの復旧のためにある程度の知識は必要。



force pushシテオキマシタ！

今回の目標

Gitで使われる用語を一通り学ぶ

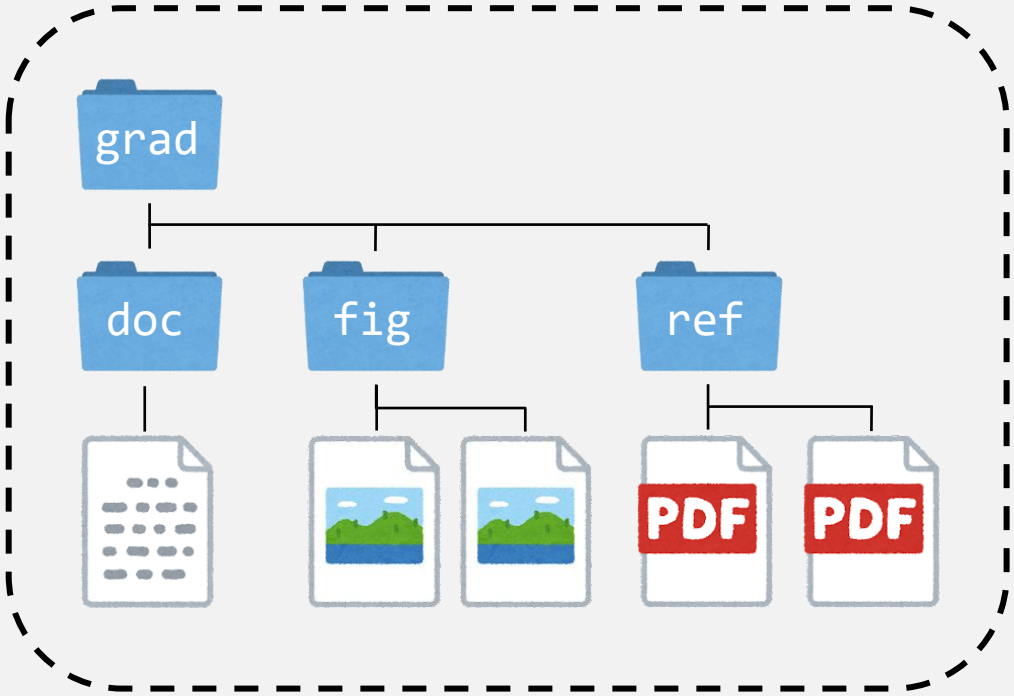
「コミット」や「ブランチ」など、バージョン管理システムでは共通の単語が使われるが、ツールによって意味や用法がかなり異なる

以下の説明はGitでの説明であることに注意

プロジェクト

管理したいファイルやフォルダの集まりを「プロジェクト」と呼ぶ

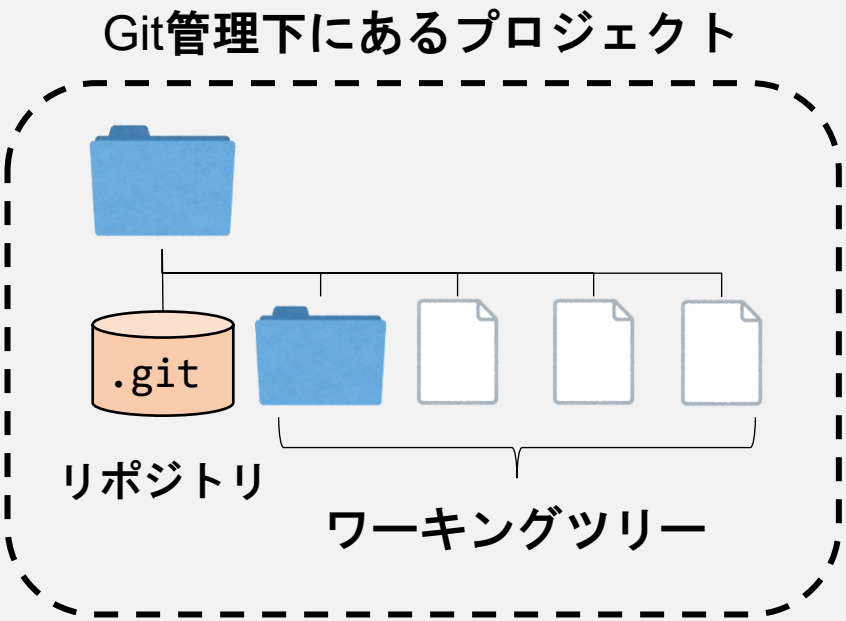
卒論プロジェクト



以後、必要なファイルを全て含むトップレベルフォルダをプロジェクトと同一視する

リポジトリとワーキングツリー

Gitで管理されたプロジェクトには.gitというフォルダがある



リポジトリ：履歴や状態を保存する場所
ワーキングツリー：作業中のファイルやフォルダ

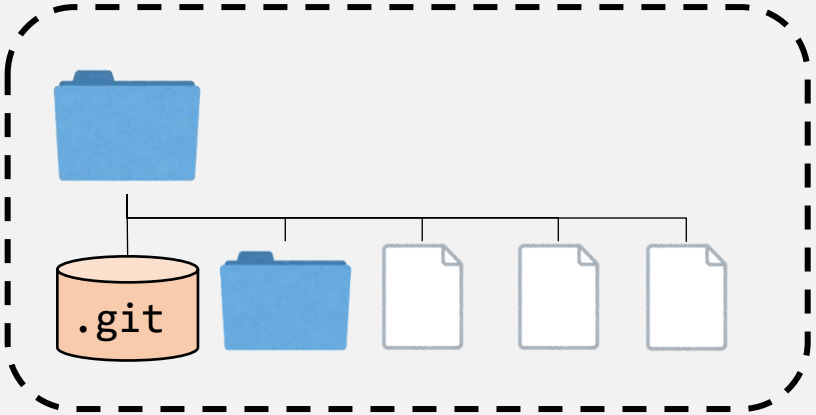
プロジェクト一つにリポジトリ一つ

リポジトリの種類

自分のPC

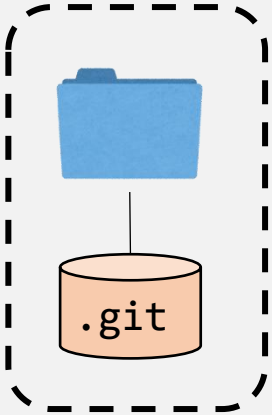
サーバ

ローカルリポジトリ



(ノンベアリポジトリ)

リモートリポジトリ



(ベアリポジトリ)

リポジトリの種類

ローカルリポジトリ (local repository)

手元のPCにあるリポジトリのこと
ワーキングツリーを持つ

リモートリポジトリ (remote repository)

リモート(ネットワークの向こう側)にある
ワーキングツリーを持たない

ベアリポジトリ (bare repository)

ワーキングツリーを持たないリポジトリ
一般にリモートはベアリポジトリ

スナップショットとコミット

スナップショット(snapshot)

ある時点でのプロジェクト全体の状態

コミット (commit)

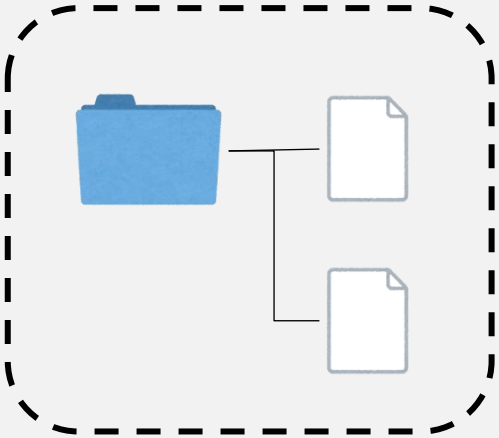
Gitにある時点でのスナップショットを登録すること
またはそのスナップショットそのもの

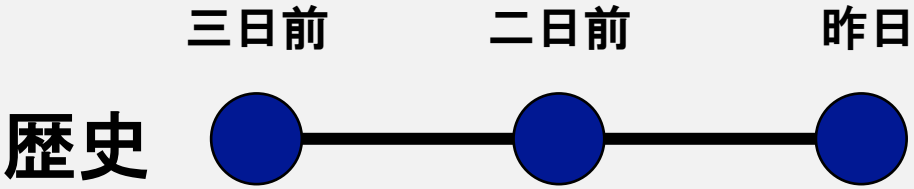
コミット

Gitでは「歴史」を丸と線で表現する

- 丸：ある時点の「状態」
- 線：二つの状態の関係(差分)

昨日から修正を加えたプロジェクト
(スナップショット)

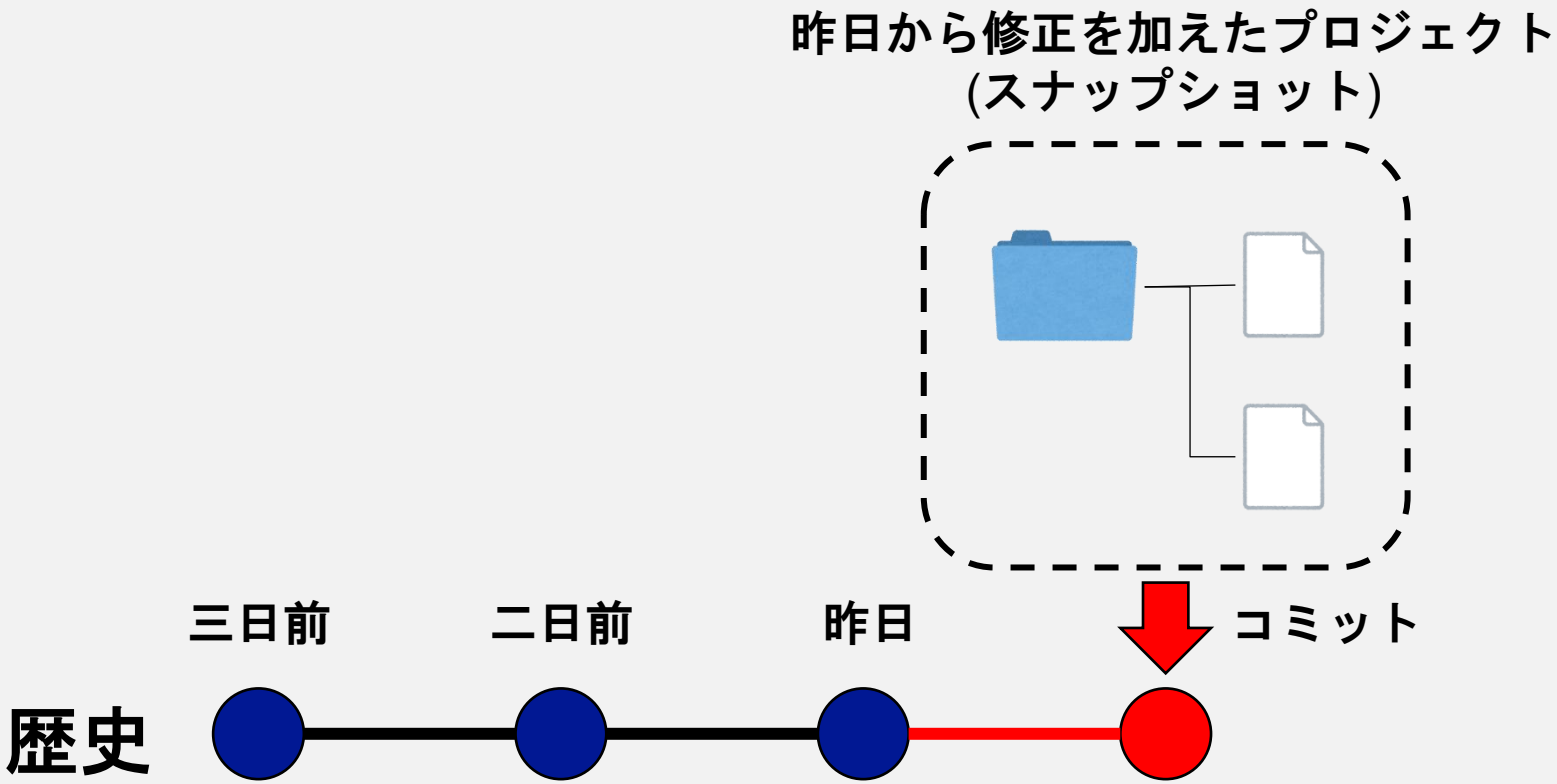




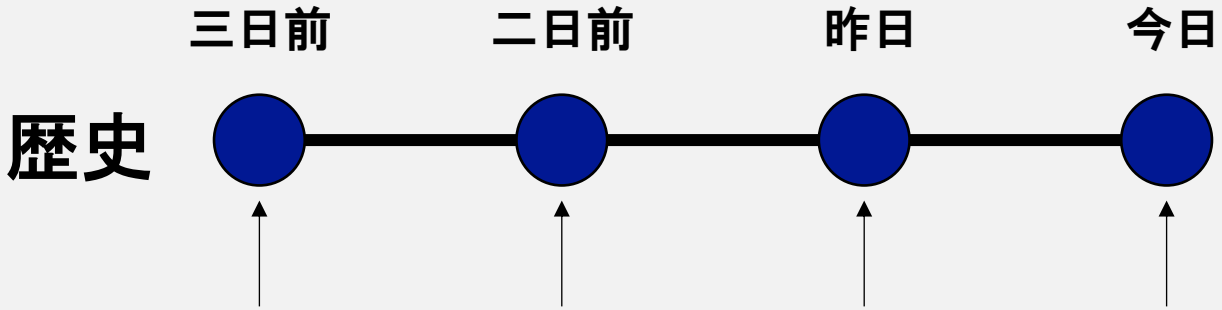
コミット

コミット：現在の状態を保存して「歴史」に加える

(スナップショット)



コミット



この玉それぞれを「コミット」と呼ぶ

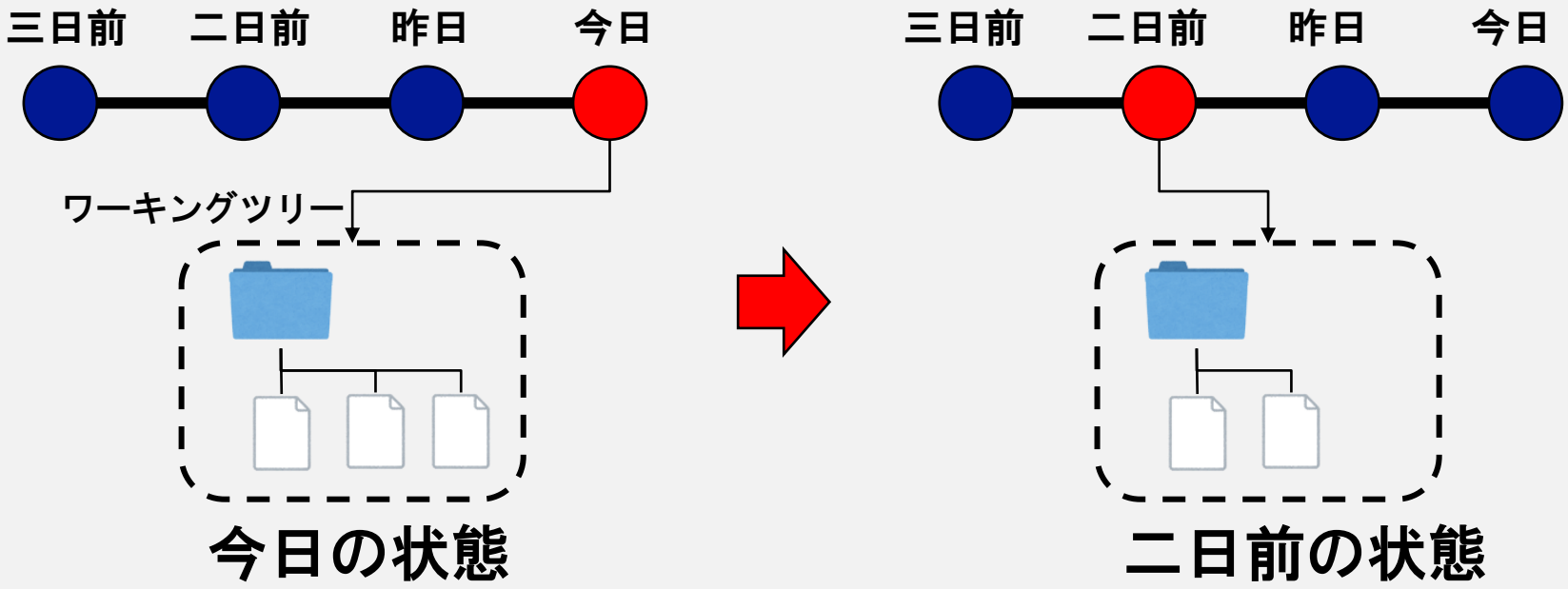
この玉を新たに作る作業を「コミットする」と呼ぶ

commit (名詞) : Gitの歴史のある「点」(スナップショット)
commit (動詞): Gitの歴史に新たにスナップショットを付け加えること

A Git Glossary https://git-scm.com/docs/gitglossary

コミット

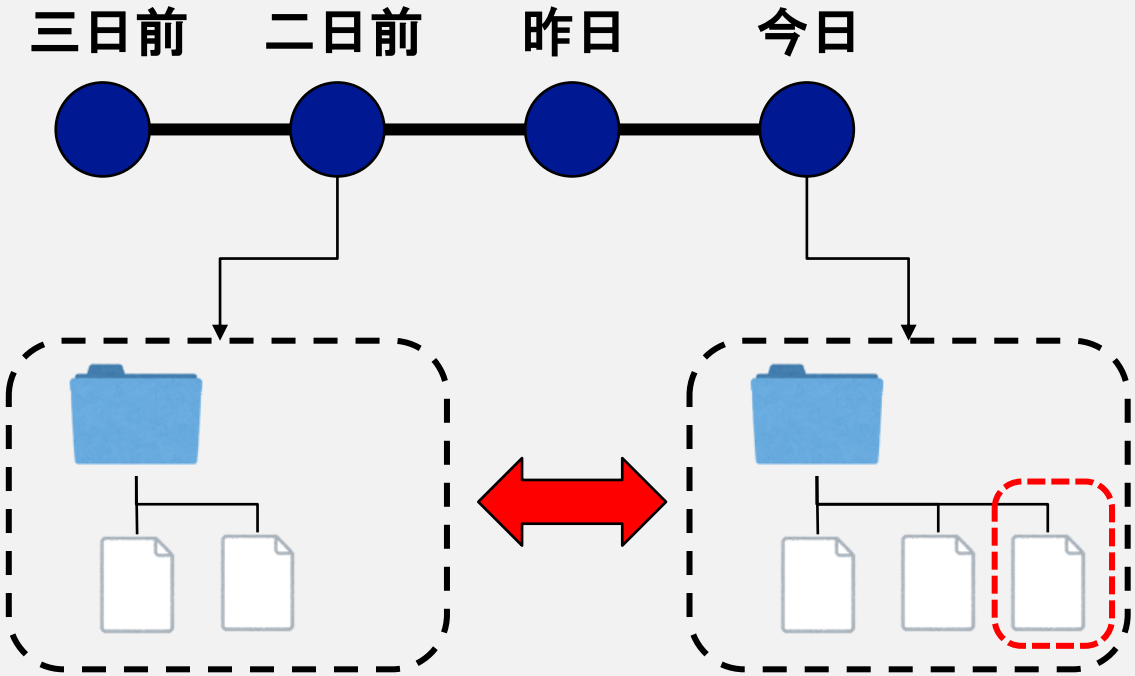
歴史上の任意の地点に戻ることができる



コミットはセーブファイルのようなもの

コミット

歴史上の任意の地点を比較できる



デバッグに極めて有用

インデックスとステージング

Gitには三種類の場所がある

ワーキングツリー(workingtree)

現在修正中のプロジェクトファイルがある場所

インデックス (index)

ステージングをする場所
コミットするスナップショットを用意する

リポジトリ (repository)

歴史(コミット)を保存する場所

インデックスとステージング

ステージング環境

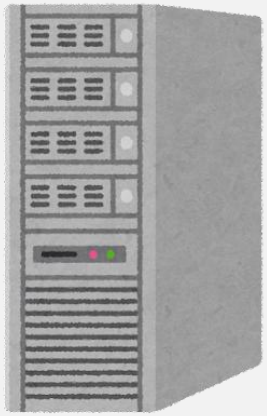
本番環境に反映させる前に準備をする場所

ステージング

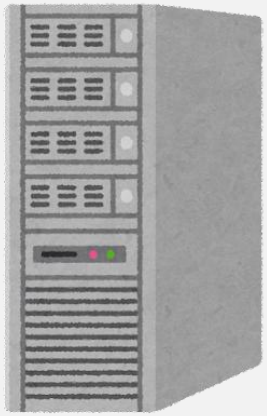
ステージング環境に修正を反映すること

ローカルPC

ステージング環境



本番環境



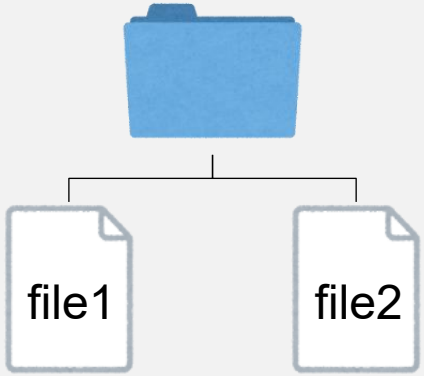
ローカルPCで修正

ここで動作テスト

本番環境へ反映

インデックスとステージング

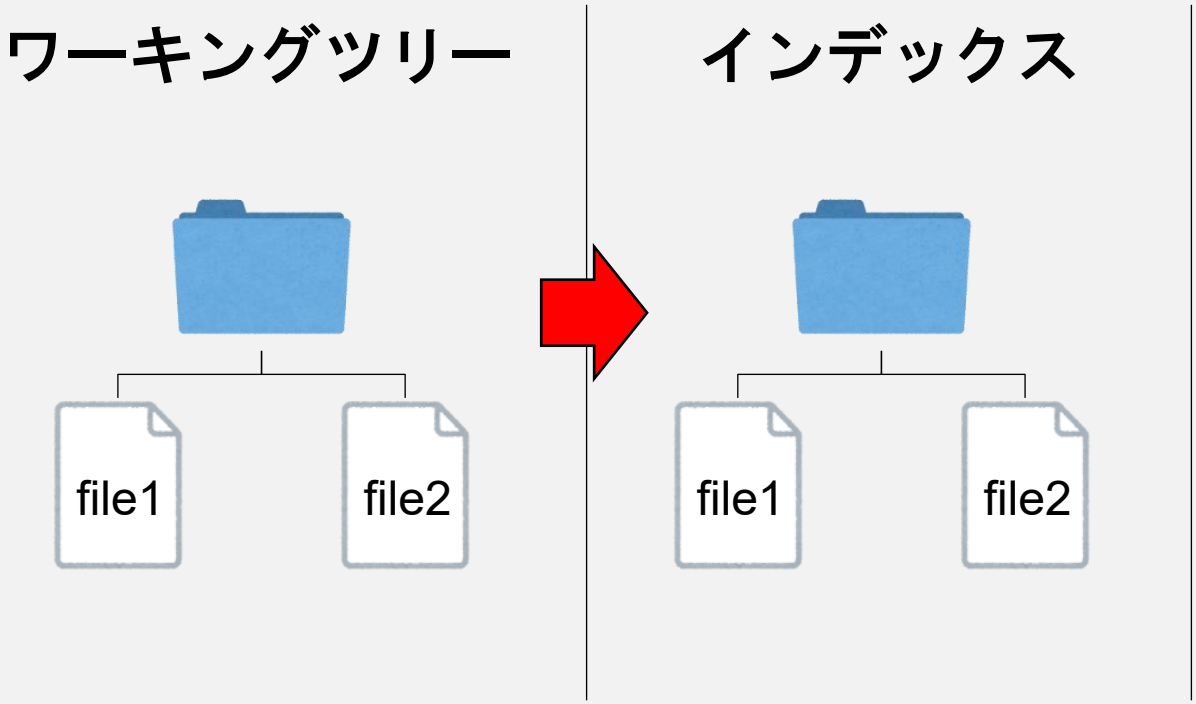
Gitの初期化直後

ワーキングツリー	インデックス	リポジトリ
 まだGit管理下に 置かれていない		

インデックスとステージング

ステージング

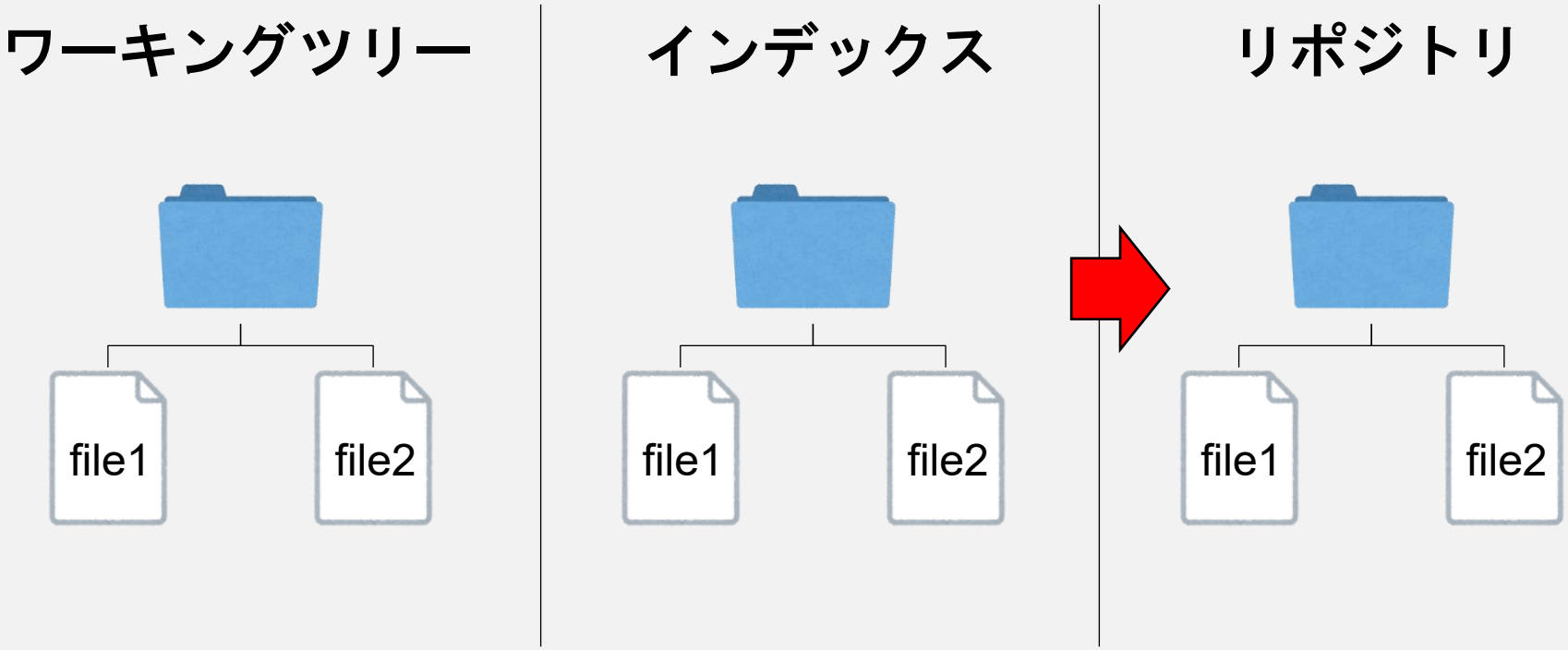
ワーキングツリーの状態(スナップショット)をインデックスに登録



インデックスとステージング

コミット

インデックスにステージングされた状態をリポジトリに登録



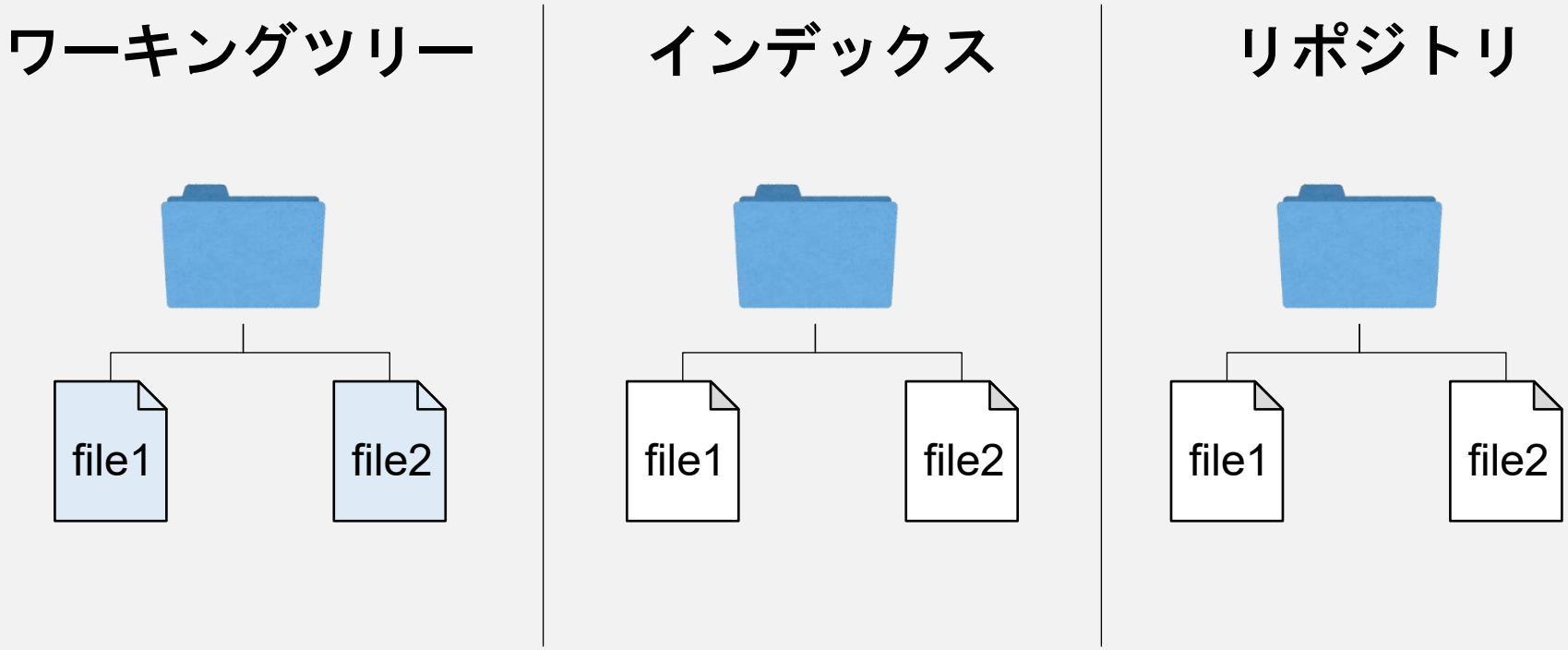
インデックスとステージング

Q: なぜインデックスがあるの？

A: 複数の修正がある時、一部の修正を選んでコミットを作るため

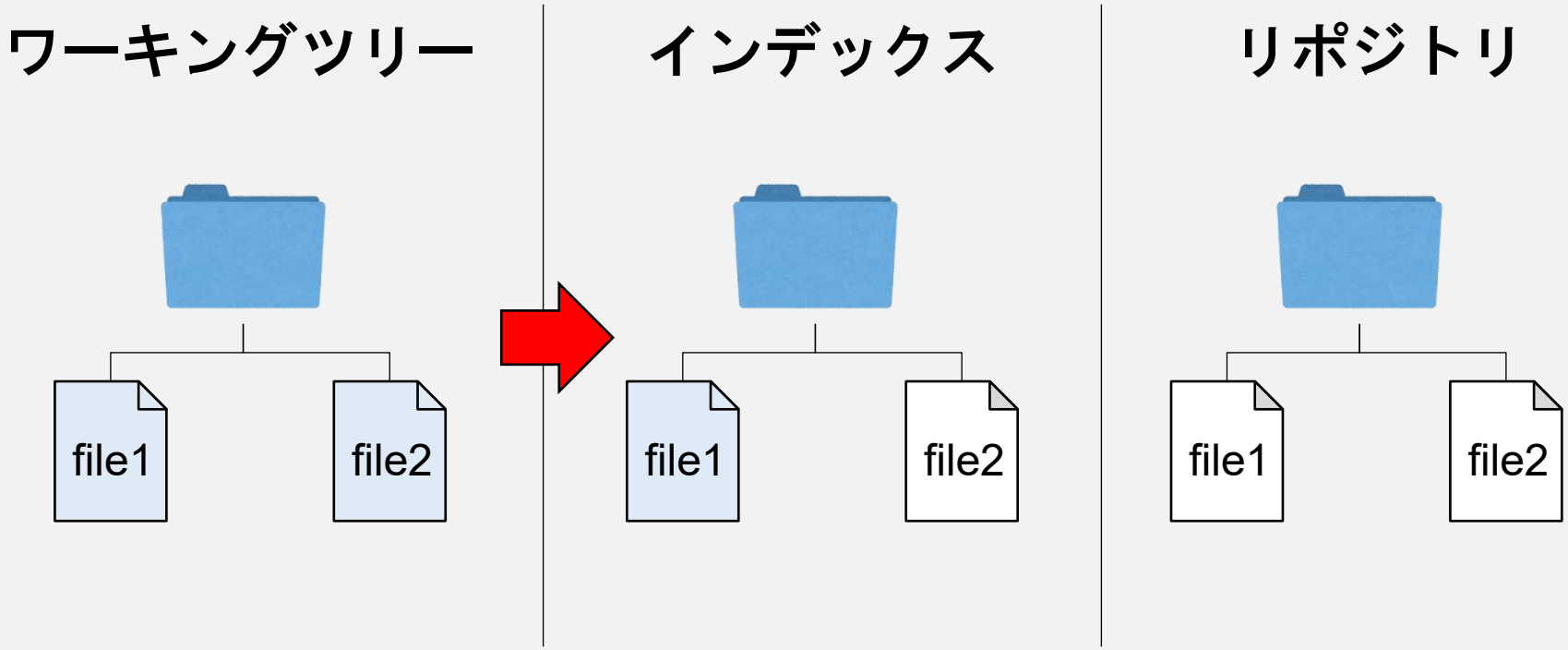
インデックスとステージング

最後にコミットした状態からfile1とfile2を修正した



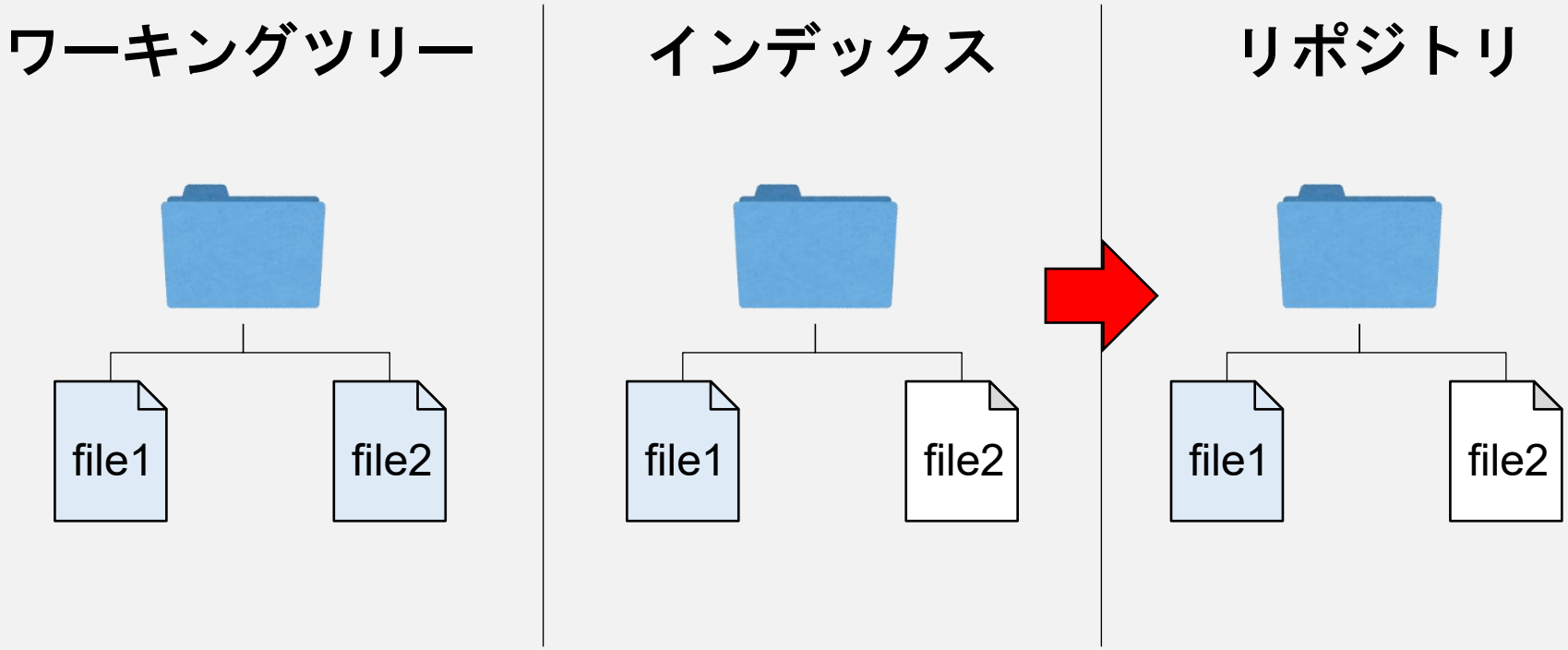
インデックスとステージング

file1だけステージングする



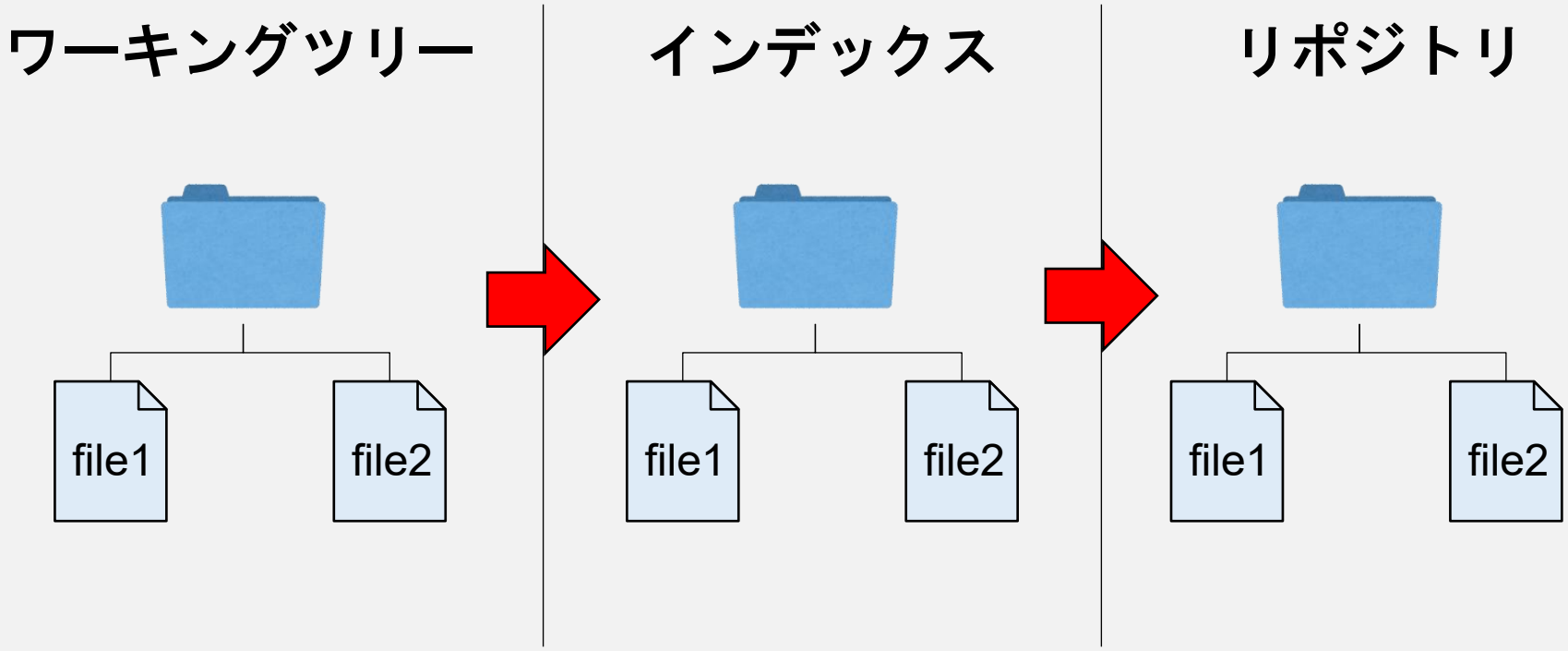
インデックスとステージング

コミットする



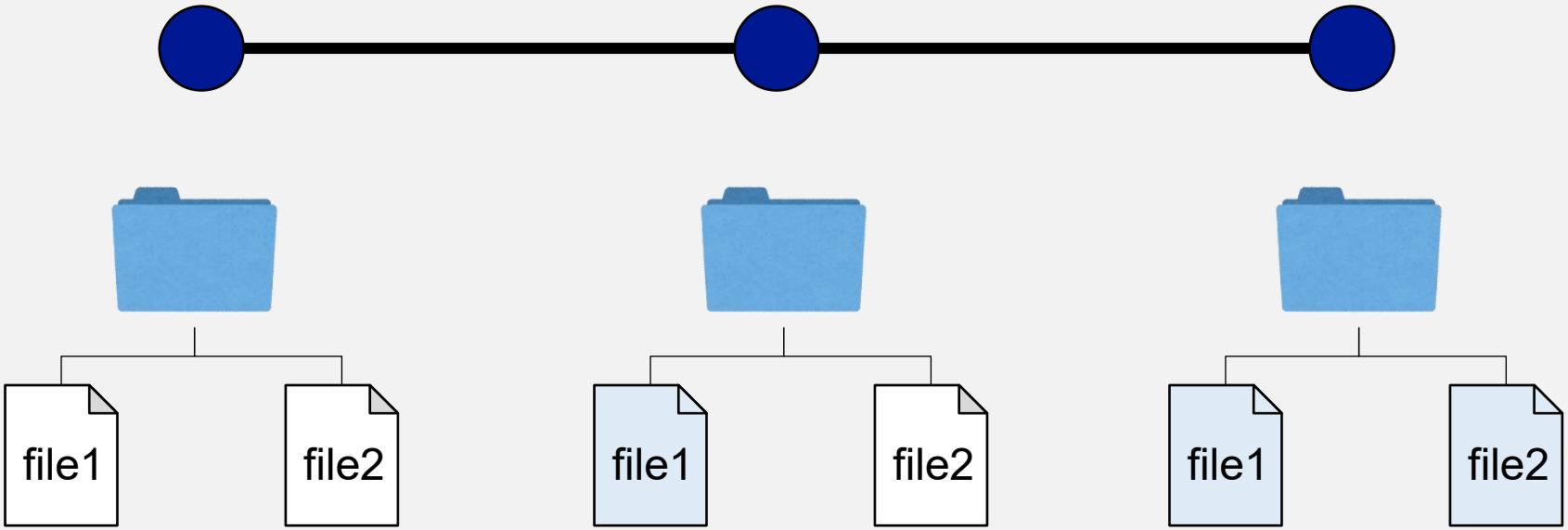
インデックスとステージング

file2も同様にステージング、コミットする



インデックスとステージング

こんな歴史ができあがった



file1とfile2を追加　　file1を修正　　file2を修正

Gitでは積極的に歴史を作成、改変する

HEADとブランチ

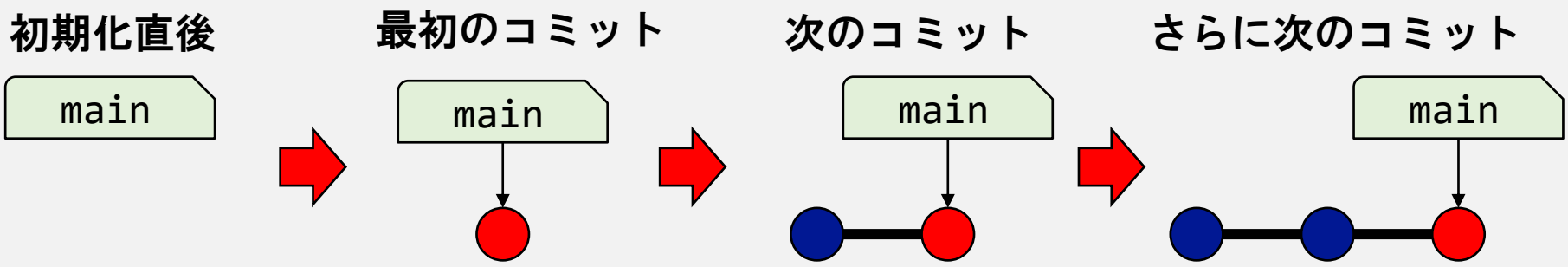
Gitでは歴史を積極的に改変する
Gitでは歴史が分岐することがある

Gitにおいて歴史操作の手段として使うのがブランチ

ブランチ

ブランチ (branch)

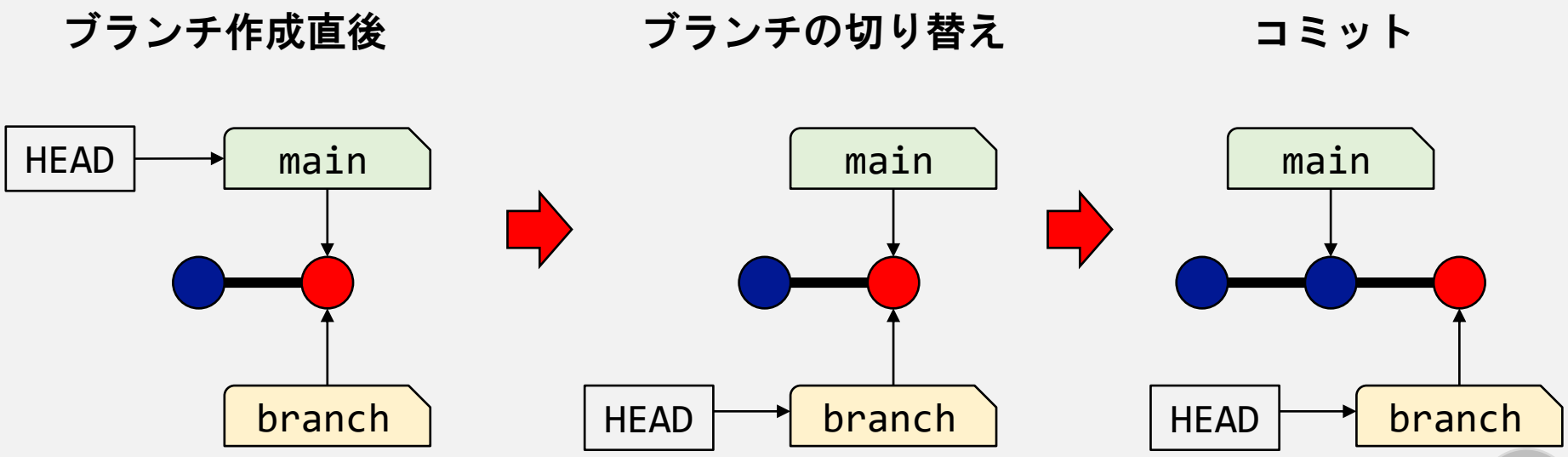
コミットにつくラベルのようなもの
デフォルトでmainというブランチが用意される
自由に作ることができる
コミットされると一緒に動く



ブランチ

カレントブランチ (current branch)

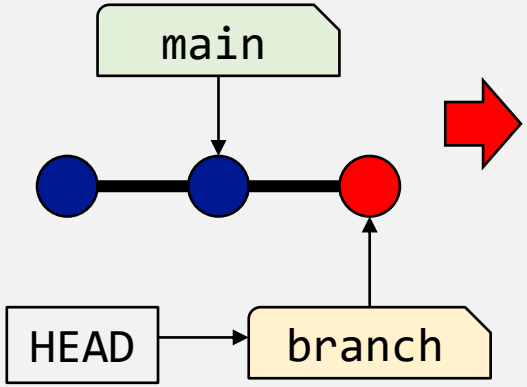
「今見ているブランチ」のこと
HEADというラベルが指す
コミットにより動くのはカレントブランチのみ



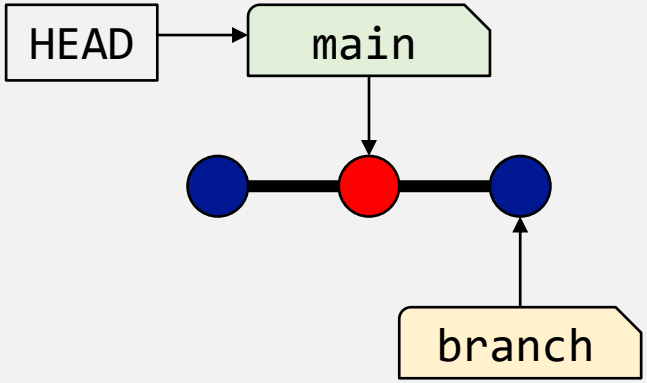
ブランチ

ブランチを切り替えながらコミットをすると
一般に歴史が分岐する

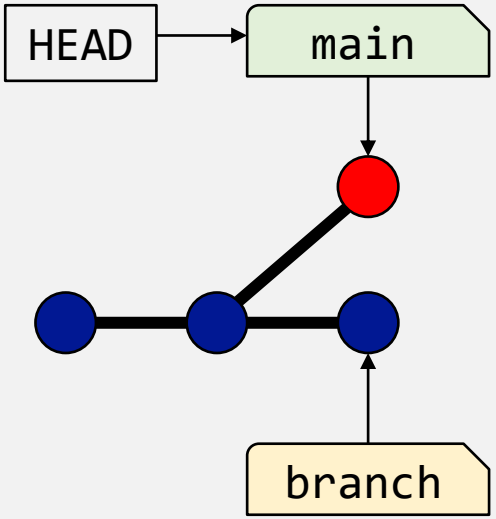
branchを
見ている状態



mainに
切り替える



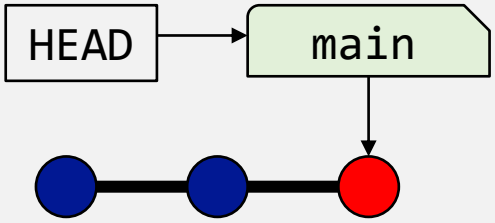
コミットする



Detached HEAD状態

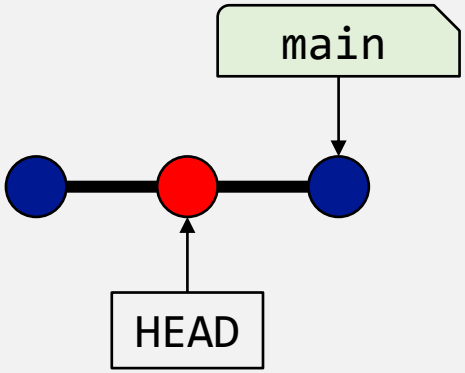
通常、HEADはブランチを指しているが、HEADが直接コミットを指す状態を**頭が取れた(detached HEAD)状態**と呼ぶ

通常の状態



HEADがブランチを経由してコミットを指している

detached HEAD状態



HEADがブランチから外れコミットを直接指している

マージ

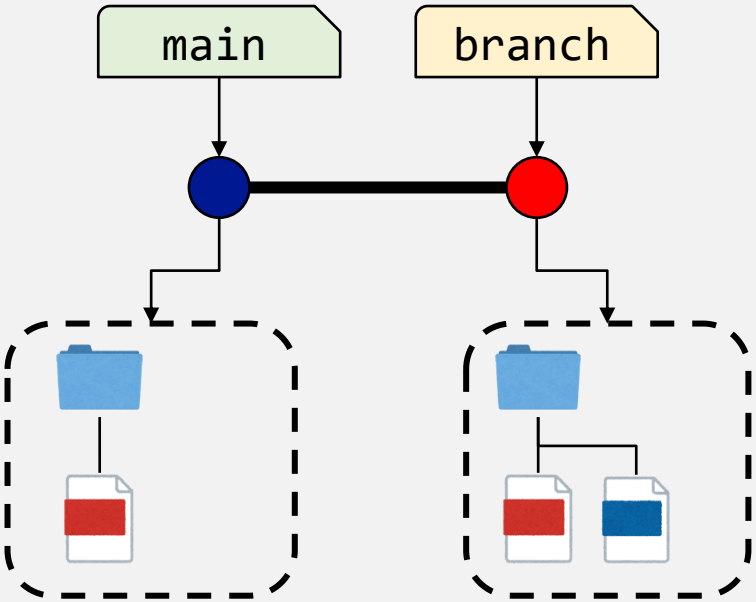
マージ(merge)

あるブランチの修正を、別のブランチに取り込むこと

Gitでは、原則としてデフォルトブランチ(main)で作業しない
別のブランチで作業し、作業が一段落したらmainに取り込む
この取り込む作業をマージと呼ぶ

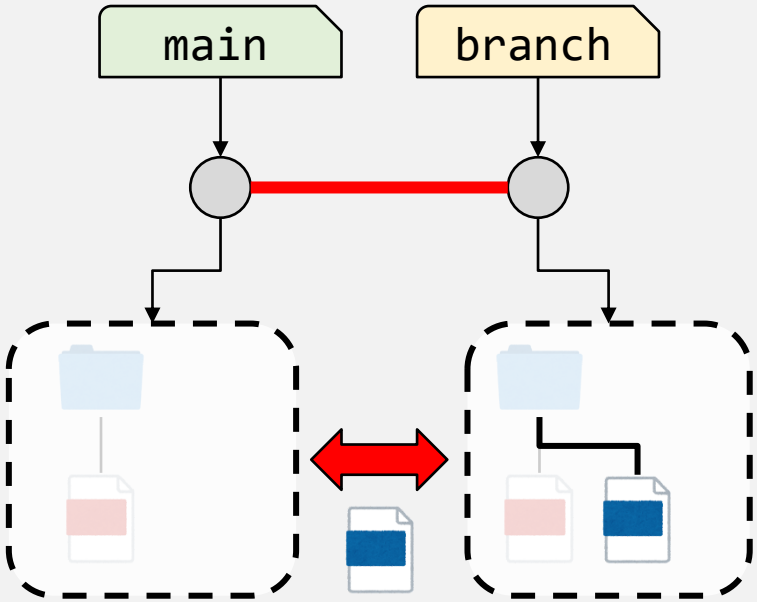
マージ

玉：スナップショット
線：スナップショット間の**差分**



mainの
スナップショット

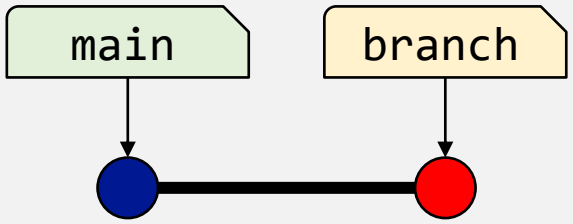
branchの
スナップショット



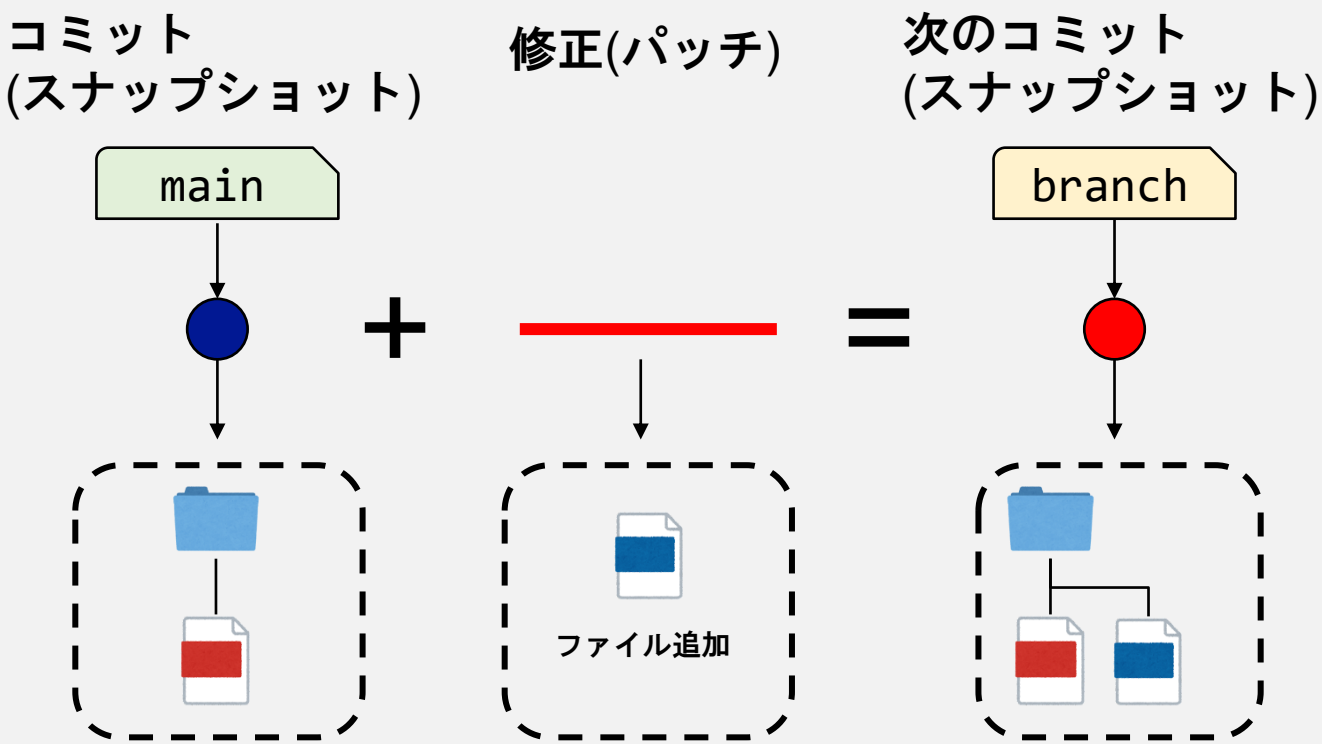
ファイル追加

mainからbranchへ更新する
ための「差分」

マージ

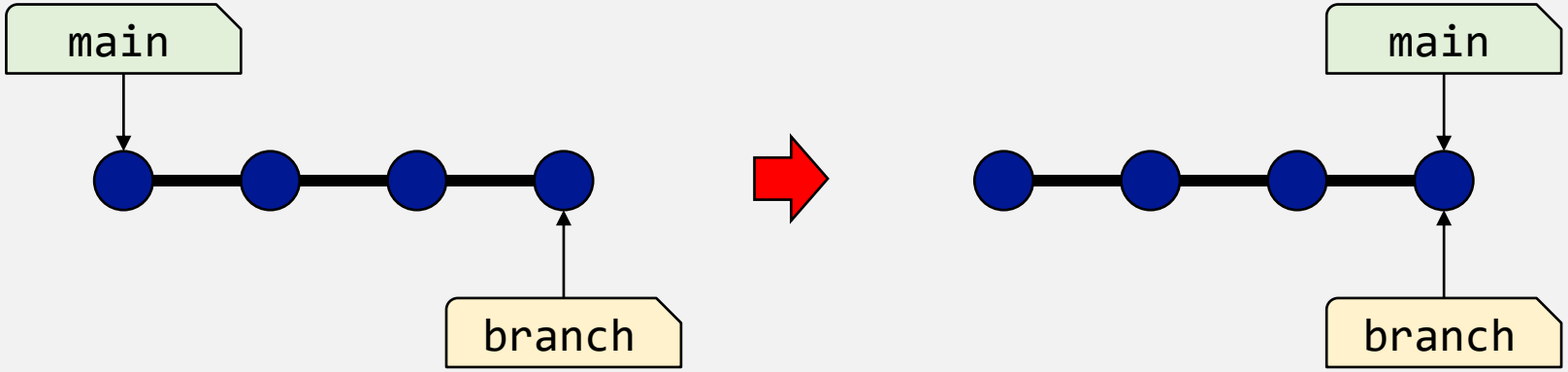


スナップショット(玉)に、パッチ(線)を適用すると、次のスナップショット(玉)になる



Fast-forwardマージ

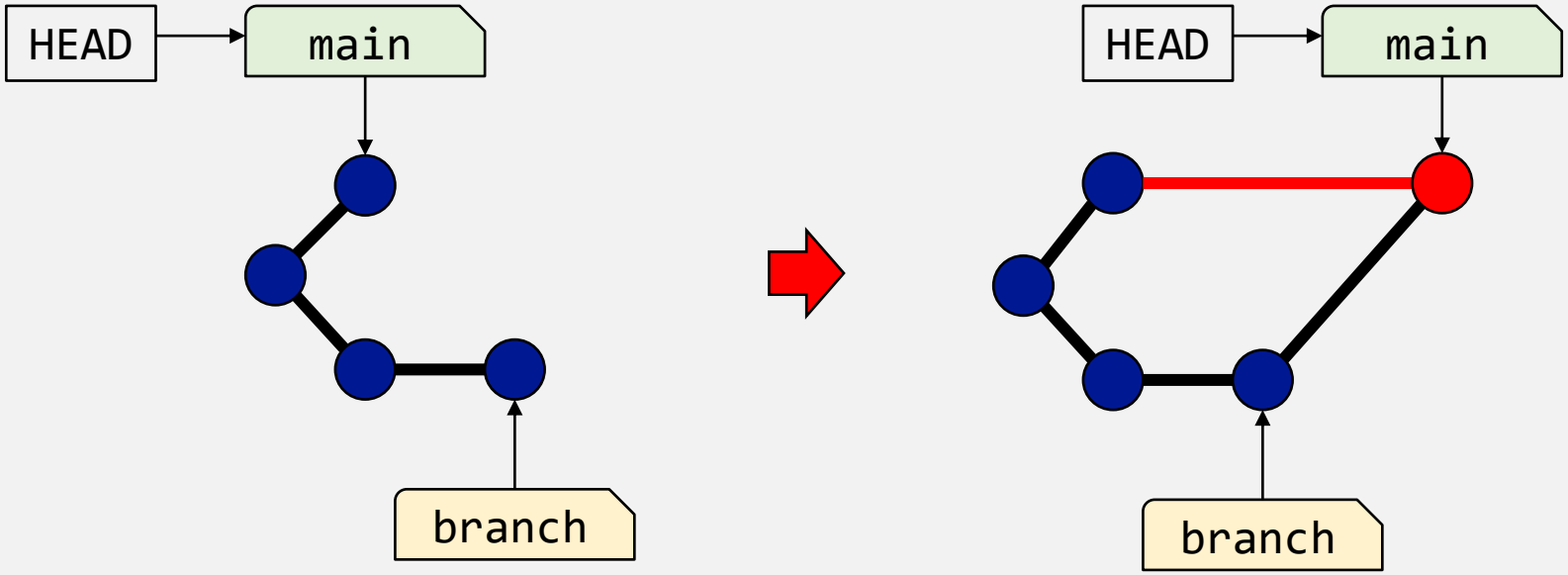
修正を取り込みたいブランチが、もう一方のブランチの直接の祖先である場合、ブランチを移動するだけでマージが完了する



これをFast-forwardマージ(早送りマージ)と呼ぶ

Non-fast forwardマージ

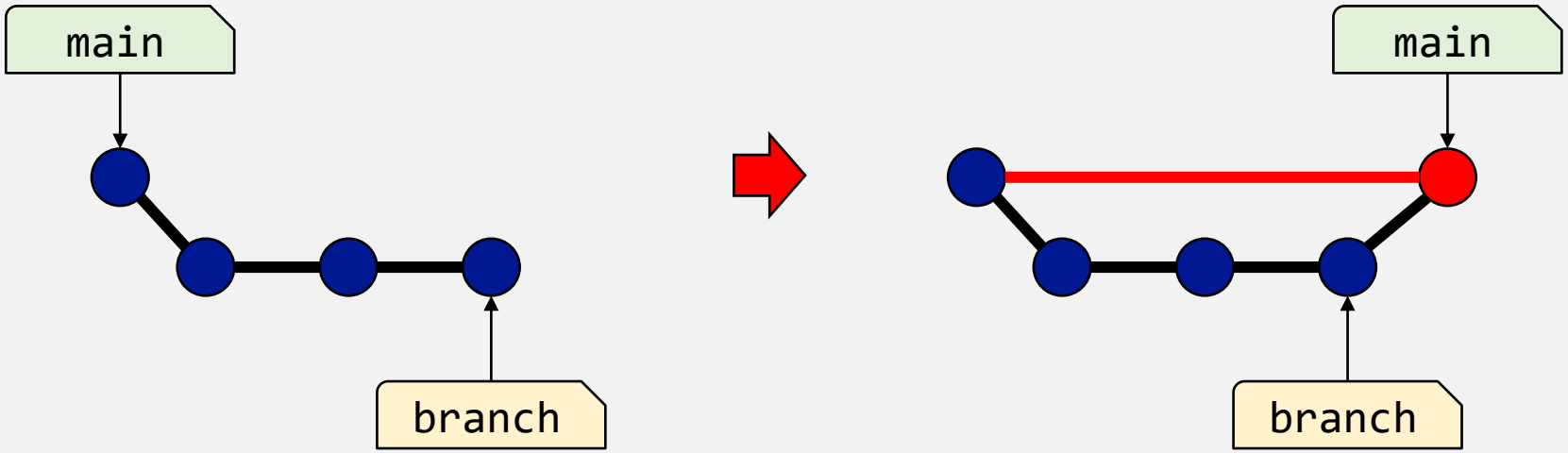
歴史が分岐している場合、マージにより歴史が合流する



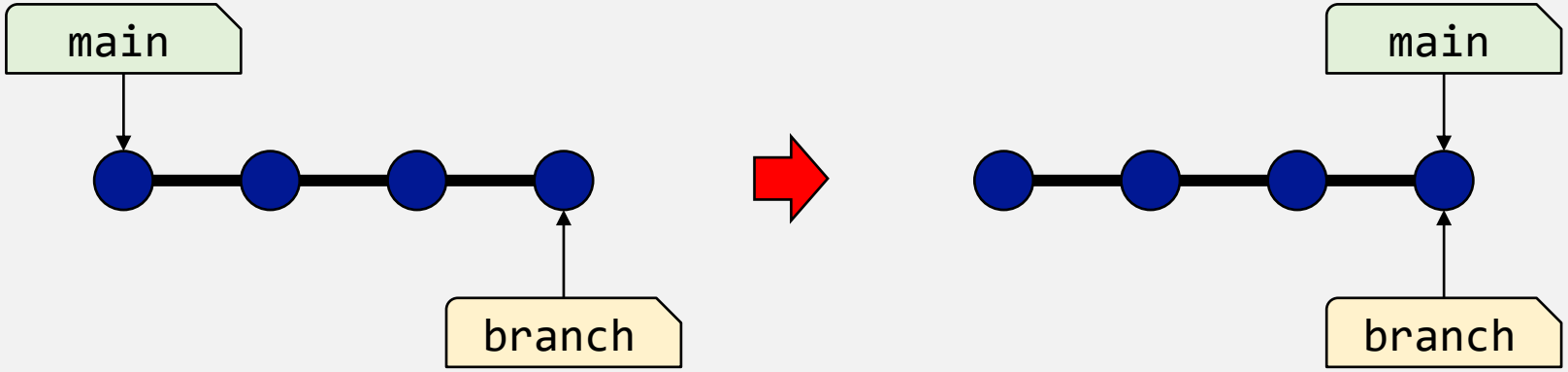
新たに作られるコミットをマージコミット(merge commit)と呼ぶ

Non-fast forwardマージ

Fast Forwardマージできる場合でもマージコミットを作ることができる



Fast forwardマージすると、マージ元がどこで分岐したかの情報が失われる



まとめ

今日ここだけは覚えて欲しい

Gitの歴史は「丸」と「線」で表現する
「丸」がスナップショット、「線」が差分を表す

コミット(名詞)：スナップショット
コミット(動詞)：スナップショットの登録
インデックス：コミットの前に修正を登録する場所
ステージング：インデックスに修正を登録すること
ブランチ：コミットについたラベル
マージ：二つのブランチの修正を一つにまとめること